CareerHubX

Assignment 3.2 — Walkthrough

Multi-step application wizard, document uploads, branding & the testing suite

Next.js 15 · React 19 · TypeScript · Tailwind v4 · TanStack Query · nuqs · Auth.js
Testing: Vitest · Testing Library · user-event · MSW · jsdom · GitHub Actions
Prepared 2026-06-30 · Branch `CareerHub-3.1`

0 · Overview & what shipped

This document walks through everything delivered in this session, how it was built, the key code, how the UI behaves, and a one-page terminal runbook (frontend + backend + Docker + tests).

Two bodies of work

Assignment 3.2 — Feature build	Assignment 3.2 — Testing
① 5-step Job Application Wizard ② Document uploads (PDF, conditional, consent) ③ CareerHubX branding ④ Backend contract / gap list	Part 1 written decisions · Part 2 Vitest setup · Part 3 wizard tests · Part 4 MSW network tests · Part 5 GitHub Actions CI

Verification status

Check	Command	Result
Type check	`npx tsc --noEmit`	**0 errors**
Production build	`npm run build`	**compiles (all routes)**
Tests	`npm run test:run`	**12 passed / 12**
CI	GitHub Actions on push/PR → main	**workflow committed**

Backend & Docker note. The live ASP.NET/PostgreSQL backend (`careerhub-api`) is a separate repository and is not in this workspace, so it could not be built or run via Docker from here. The testing assignment does not need it (tests use MSW + jsdom). Where the new features need persistence the frontend is wired to the *real contract names* and falls back to `localStorage` (clearly flagged) so the flow is fully functional for a demo. The exact endpoints/DTOs the backend must add are specified in `docs/BACKEND-GAPS.md` . The terminal runbook (§6) includes the Docker commands for when the backend repo is present.

1 · The 5-step application wizard

Feature 1 `src/components/apply/JobApplicationWizard.tsx` · route `/apply/[jobId]`

What it does

#	Step	Collects
1	Personal information	full name(s), surname, gender, race (optional + "Prefer not to say"), date of birth, disability (+details if Yes), nationality
2	Contact details	email, phone, alternate phone, address, city, province (SA + Other), postal code
3	Qualifications	highest qualification, institution, year, employment status, years of experience, LinkedIn/portfolio
4	Job-specific	motivation (≥100 chars), expected salary (ZAR), notice period, start date, willing to relocate
5	Documents & consent	required PDFs (§2) + three consent checkboxes

How it's built

- **One Zod schema** (`src/lib/apply-wizard.ts`) with per-step field lists; **React Hook Form** validates only the current step before Next.
- **nuqs** keeps position in the URL (`?step=`) so a refresh resumes there.
- **Save as Draft** on every step (localStorage fallback) + profile pre-fill.
- **Accessible progress bar** (`WizardProgress.tsx`): `aria-current="step"` , visited steps are keyboard-focusable buttons, collapses to "Step N of M" under `md` .
- **Submit** → AlertDialog confirm → `sonner` toast → redirect to a printable confirmation page.

Key code — per-step validation gate

```
// trigger() validates ONLY the current step's fields before advancing
async function next() {
  const valid = await trigger(STEP_FIELDS[current], { shouldFocus: true });
  if (!valid) return;        // blocked — errors render inline
  persistDraft(current + 1);    // Save-as-Draft semantics on every advance
  goto(current + 1);
}
```

Key code — URL as a durable mirror (robust + testable)

```
const [urlStep, setUrlStep] = useQueryState("step",
  parseAsInteger.withDefault(0).withOptions({ history: "replace" }));
const [current, setCurrent] = useState(() => clampStep(urlStep ?? 0)); // read once
function goto(target) { setCurrent(clampStep(target)); void setUrlStep(target); } // write on change
```

2 · Documents step (PDF uploads + consent)

Feature 2 `src/components/apply/DocumentUpload.tsx`

How the UI behaves

"No special characters or spaces in filenames — uploads are renamed automatically to `{userId}-{document}.pdf`."

- Drag-and-drop *or* click; **PDF only, max 3 MB**, validated with **inline** errors (not toasts).
- Display name is server-style sanitised; size shown in KB; **Remove** is guarded by an AlertDialog.

Conditional document matrix

Document	Required	Condition
ID Document (SA) *or* Passport	Yes	by nationality (step 1)
Matric Results	Yes	always
Tertiary Qualification	Conditional	if step-3 qualification is tertiary
Driver's Licence	Conditional	if the job requires it
Motivation Letter / CV	Yes	always

Key code — required-set is derived from answers

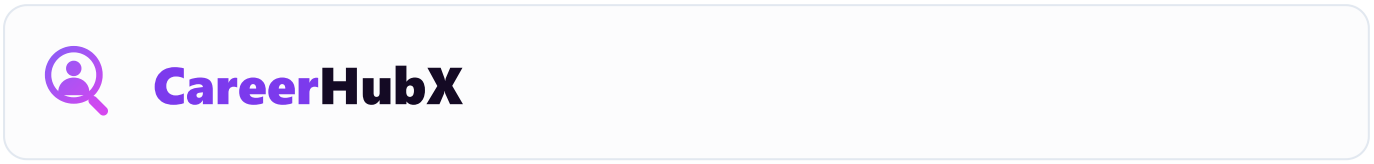
```
const reqDocs = requiredDocuments({
  isSouthAfrican: nationality === "South African",
  hasTertiary: isTertiary(highestQualification),
  jobRequiresLicence: job.requiresDriversLicence,
});
const allDocsUploaded = reqDocs.every((d) => docs[d.type]);
// Submit is disabled until allDocsUploaded && all three consents are ticked.
```

> Consent (all required before submit): *information is accurate · consent to data processing per the Privacy Policy · agree to be contacted by employers.* A `/privacy` page was added for the link.

3 · CareerHubX branding

Feature 3 `src/components/brand/Logo.tsx` · `src/app/icon.svg`

An inline-SVG brand mark (a person silhouette inside a magnifying-glass ring) and a two-tone “CareerHubX” wordmark, scaling and theming via brand tokens — no PNG.



- `<Logo>` (mark + wordmark, `onDark` variant) and `<LogoMark>` (icon only).
- Wired into the navbar, footer, auth shell, and the `app/icon.svg` favicon.
- Page titles use a `%s · CareerHubX` template + OpenGraph defaults in `layout.tsx`.

4 · Testing — setup & the adaptation

> **The key insight.** The assignment targets the *old* 3-step wizard (`useSession`, HTTP submit, a "review" step). This session had replaced it with the 5-step wizard (auth as a `user` prop, **localStorage** submit, separate confirmation page). So the demo pattern was **adapted to the real components** — which the brief explicitly rewards.

Assignment expectation	Real component tested
Wizard navigation / validation	`JobApplicationWizard`
Auth gate	`/apply/[jobId]` page (gating moved to the page)
MSW submit (happy / error)	`ApplicationForm` (the component that actually POSTs + resets)
AlertDialog confirm	`CloseJobButton` (Server Action mocked — MSW can't intercept actions)
Draft persistence	`JobApplicationWizard` via real jsdom localStorage

Part 2 — the provider harness

```
// src/test/utils.tsx
vi.mock("next-auth/react", () => ({ useSession: vi.fn(), /* … */ }));
export function renderWithProviders(ui, { session = candidateSession, searchParams = "" } = {}) {
  vi.mocked(useSession).mockReturnValue({ data: session, status: session ? "authenticated" : "unauthenticated"
  return render(ui, { wrapper: ({ children }) =>
    <NuqsTestingAdapter searchParams={searchParams}>
      <QueryClientProvider client={makeClient()}>/* retry:false */
        <AuthProvider>{children}</AuthProvider>
      </QueryClientProvider>
    </NuqsTestingAdapter> });
}
```

Part 4 — MSW: real HTTP interception

```
// happy path returns 201; error path overrides per-test:
server.use(http.post(`${API}/api/applications`, () => new HttpResponse(null, { status: 500 })));
// → assert a server-error alert appears AND the typed value is still present (no reset)
expect(await screen.findByRole("alert")).toBeInTheDocument();
expect(screen.getByDisplayValue("Alice Candidate")).toBeInTheDocument();
```

5 · Testing — results & decisions

Suite (12 tests, 5 files) — all green

File	Tests
`ApplicationWizard.test.tsx`	smoke · blocks-when-invalid · advances to step 2 · Back preserves values
`apply-gate.test.tsx`	sign-in prompt when unauthenticated · wizard renders for a candidate
`ApplicationForm.test.tsx`	MSW happy path (confirmation) · MSW 500 (values kept)
`CloseJobButton.test.tsx`	AlertDialog opens · confirm runs the action + shows closed state
`draft-persistence.test.tsx`	restores a saved draft + banner · writes a draft on "Save as Draft"

Written decisions (condensed — full text in README)

Q1 high-value: step gating, Back-preserves-values, document/consent gate, submit success/error. **NOT worth testing:** Tailwind class strings & DOM shape — they break on restyles and assert nothing the user feels.

Q1C / localStorage: use **real jsdom** localStorage — the behaviour *is* the write→reload→restore round-trip; a spy only proves a method was called, not that the value comes back into the form.

Q2 session: `vi.mock("next-auth/react")` (mocks just the hook, keeps the tree real). Caveat: our components read `AuthContext`, so the gate test plants `careerhub.session` in localStorage.

Q3 MSW scope: GET job, POST application, GET jobs (post-submit invalidation). **MSW cannot test** the wizard's own submit (localStorage) or `CloseJobButton` (Server Action) — neither is a network request.

Q4 naming: rewrote implementation-named tests (e.g. "currentStep equals 'schedule'", "calls setItem with key", "renders 3 divs") into behaviour names ("advances to the schedule step", "a returning user resumes where they left off").

> **The test that surprised me.** "Advance to step 2" flaked only under full-suite load. It revealed that the wizard's `Next` handler validates *asynchronously* and updates the step *after* the `await`, so the re-render lands outside Testing Library's `act()` window. The fix was in the test (a `clickNext` helper that flushes the continuation inside `act`), not the product — a gap in my mental model of async handlers + `act`, not a bug.

6 · PowerShell runbook — every command to run it

Copy-paste, top to bottom, from the project root in **PowerShell**.

A · Frontend (this repo)

```
# install dependencies (first time only)
npm install

# run the dev server → http://localhost:3000
npm run dev

# production build + start (verifies the whole app compiles)
npm run build ; if ($?) { npm run start }
```

B · Backend + database via Docker (in the careerhub-api repo)

```
# from the careerhub-api repository root (NOT this frontend repo):
docker compose up --build          # API on http://localhost:8080 + PostgreSQL
docker compose ps                  # confirm both containers are healthy
docker compose logs -f api         # follow API logs
docker compose down                # stop everything
```

> **The backend is a separate repo, not in this workspace. Point the frontend at it with `NEXT_PUBLIC_API_BASE_URL=http://localhost:8080` in `.env.local`, then restart `npm run dev`.**

C · Tests

```
# run once and exit (what CI runs)
npm run test:run

# watch mode while developing
npm test

# a single file
npx vitest run src/test/ApplicationWizard.test.tsx

# type-check (zero errors expected)
npx tsc --noEmit
```

D · Prove it changed (assignment's "Proving It Changed")

```
# 1) remove the validation guard in next() → the "blocks advancement" test fails
# 2) override the POST handler to 500 in the happy-path test → it fails
# 3) comment out afterEach(() => server.resetHandlers()) → the 500 leaks between tests
# restore each → green again
```

E · CI (GitHub Actions)

```
git add . ; git commit -m "test: ci" ; git push     # Actions tab shows a run
```

```
# Workflow: .github/workflows/test.yml — push/PR to main, Node 20, npm ci, npm run test:run
```